

TASK 2

Api_client.py

```
import requests
```

```
API_BASE_URL = "https://hr-internal.company.com/api"
```

```
API_TOKEN = "your_token_here"
```

```
def get_employee(employee_id):
```

```
    url = f"{API_BASE_URL}/employees/{employee_id}"
```

```
    # Fix A: Added Authorization header
```

```
    response = requests.get(url, headers = {  
        "Authorization": f"Bearer {API_TOKEN}"  
    })
```

```
    # Fix B: Return None
```

```
    if response.status_code == 404:
```

```
        return None
```

```
    data = response.json()
```

```
    return data["employee"]
```

db_queries.py

```
def get_monthly_attendance(conn, employee_id, month, year):
```

```
    query = """
```

```
        SELECT a.date, a.check_in, a.check_out, e.name
```

```
        FROM attendance a
```

```

LEFT JOIN employees e
ON a.employee_id = e.id
WHERE a.month = ?
AND a.year = ?
ORDER BY a.date ASC
"""

cursor = conn.cursor()
# Fix B: Pass employee_id
cursor.execute(query, (employee_id, month, year))
return cursor.fetchall()

```

attendance.py

```

from datetime import datetime

WORK_START = "09:00"
LATE_THRESHOLD_MINUTES = 15

def calculate_hours(check_in: str, check_out: str):
    fmt = "%H:%M"
    ci = datetime.strptime(check_in, fmt)
    co = datetime.strptime(check_out, fmt)

    # Fix A: co-ci
    duration = co - ci
    hours_worked = duration.seconds / 3600

    start = datetime.strptime(WORK_START, fmt)
    late_by = (ci - start).seconds // 60

    # Fix B: flag the employee who is exactly 15 minutes late

```

```
is_late = late_by >= LATE_THRESHOLD_MINUTES
```

```
return round(hours_worked, 2), is_late
```

report.py

```
#Q4
```

```
def generate_summary(records):
```

```
    total_hours = 0
```

```
    late_days = 0
```

```
    # Fix A: Iterate till last index only
```

```
    for i in range(len(records)):
```

```
        record = records[i]
```

```
        total_hours += record["hours"]
```

```
        if record["is_late"]:
```

```
            late_days += 1
```

```
    # Fix B: Prevent division by zero
```

```
    avg_hours = total_hours / len(records) if len(records) > 0 else 0
```

```
    return {
```

```
        "total_hours": round(total_hours, 2),
```

```
        "avg_hours": round(avg_hours, 2),
```

```
        "late_days": late_days,
```

```
        "days_present": len(records)
```

```
    }
```

```
#Q5
```

```
MIN_DAYS_REQUIRED = 20
```

```
MAX_LATE_DAYS = 3
```

```
def check_attendance_policy(summary):  
    days_present = summary["days_present"]  
    late_days = summary["late_days"]  
  
    # Fix A:  
    below_minimum = days_present < MIN_DAYS_REQUIRED  
  
    # Fix B:  
    exceeded_late = late_days >= MAX_LATE_DAYS  
  
    if below_minimum or exceeded_late:  
        return {  
            "warning": True,  
            "reason": []  
            + (["Below minimum attendance"] if below_minimum else [])  
            + (["Exceeded late check-ins"] if exceeded_late else [])  
        }  
  
    return {  
        "warning": False,  
        "reason": []  
    }
```